HABIT HERO

AI-Powered Habit Tracking and Personal Growth Platform

Complete Software Project Documentation

Document Type: Technical Project Documentation

Audience: Academic submission, technical reviews, interviews

Documentation Date: 2026-11-11

Repository: https://github.com/vishnuk29072004-prog/Habit-Hero-

Primary Branch: main

Stack: React + Vite frontend · FastAPI + SQLite backend · Google Gemini · Google Calendar · ReportLab

This document describes the actual implemented system. Secrets, API keys and .env values are intentionally excluded.

Table of Contents

The table of contents below is a Word TOC field. In Microsoft Word, right-click and choose Update Field → Update entire table to refresh page numbers.

Table of Contents 2
1. Project Introduction 6
1.1 Introduction 6
1.2 Background 6
1.3 Problem Statement 6
1.4 Project Objectives 6
1.5 Scope 6
1.6 Target Users 7
1.7 Proposed Solution 7
1.8 Key Differentiators 7
1.9 Expected Outcomes 7
2. Complete Technology Stack 8
2.1 Frontend Technologies 8
2.2 Backend Technologies 8
2.3 Development Tools (verified usage) 9
3. System Architecture 10
3.1 Overview 10
3.2 Frontend Architecture 10
3.3 Backend Architecture 10
3.4 Folder Structure (project root) 11
3.5 External Integrations 11
4. Complete Frontend Pages 12
4.1 Dashboard (`/`) — Dashboard.jsx 12
Features 12
Backend APIs used 12
Benefits 12
4.2 Habit Management (`/habits`) — Habits.jsx 12
Features 12
Backend APIs used 12
Benefits 12
4.3 Calendar (`/calendar`) — Calendar.jsx 12
Features 13

Backend APIs used 13
Benefits 13
4.4 Analytics (`/analytics`) — Analytics.jsx 13
Features 13
Backend APIs used 13
Benefits 13
4.5 Goals (`/goals`) — Goals.jsx 13
Features 13
Backend APIs used 13
Benefits 14
4.6 Journal (`/journal`) — Journal.jsx 14
Features 14
Backend APIs used 14
Benefits 14
4.7 AI Insights (`/ai-insights`) — AIInsights.jsx 14
Features 14
Backend APIs used 14
Benefits 14
4.8 Settings (`/settings`) — Settings.jsx 14
Features 14
Backend APIs used 15
Benefits 15
4.9 Profile (`/profile`) — Profile.jsx 15
Features 15
Backend APIs used 15
Benefits 15
4.10 Authentication — Login.jsx / Signup.jsx 15
Features 15
Backend APIs used 15
Benefits 15
4.11 Unrouted page note 16
5. Backend Modules 17
5.1 Cross-cutting concerns 17
6. Complete REST API Documentation 18
6.1 Endpoint catalogue 18
6.2 Representative AI schemas (Pydantic) 19

6.3 Error handling patterns 19
7. Google Gemini AI Integration 20
7.1 AI Habit Suggestions 20
7.2 AI Journal Mood Analysis 21
7.3 Configuration (names only) 21
8. Google Calendar Integration 22
8.1 Setup requirements 22
8.2 Environment variable names 22
8.3 Runtime behavior 23
9. PDF Report Generation 24
9.1 Report sections (from code) 24
10. Database Design 25
10.1 ORM interaction 26
11. Authentication and Security 27
11.1 Example environment placeholders 27
12. Habit Tracking Algorithms 28
12.1 Scheduling 28
12.2 Day status 28
12.3 Completion percentage 28
12.4 Streaks 28
12.5 Example 29
13. Notifications and Reminders 30
14. Complete User Workflow 31
15. Project Advantages 32
16. Testing and Validation 33
16.1 Backend pytest modules 33
16.2 Distinguishing test types 33
16.3 Known testing limitations 33
17. Installation and Execution (Windows PowerShell) 34
17.1 Prerequisites 34
17.2 Backend 34
17.3 Frontend 34
17.4 Troubleshooting 34
18. GitHub and Version Control 35
19. Limitations 36
20. Future Enhancements (not implemented) 37

21. Conclusion 38
22. Appendices 39
Appendix A — Diagram index 39
Appendix B — Key source paths 39
Appendix C — References 39
Appendix D — Verification statement 39

(Update TOC field in Microsoft Word to populate entries.)

1. Project Introduction

1.1 Introduction

Habit Hero is a full-stack web application for building consistent habits, tracking daily and weekly completions, managing personal goals, journaling reflections, and generating AI-assisted insights. The product pairs a React single-page application with a FastAPI backend, SQLite persistence, JWT authentication, Google Gemini AI features, ReportLab PDF export, and optional Google Calendar synchronization.

1.2 Background

Personal productivity tools often separate habit tracking, journaling, analytics and calendar planning into disconnected apps. Habit Hero consolidates these concerns into one authenticated workspace where the application remains the source of truth for habits, while external services (Gemini, Google Calendar, PDF generation) are invoked on demand.

1.3 Problem Statement

- Users struggle to maintain consistent habit schedules across daily and weekly cadences.
- Progress is hard to visualize without streak, category and calendar analytics.
- Generic habit suggestions rarely account for a user's existing habits and goals.
- Journal writing lacks structured reflection on mood and motivation patterns.
- Exportable progress reports and calendar visibility are frequently missing or insecurely integrated.

1.4 Project Objectives

1. Provide secure multi-user habit tracking with JWT access and refresh tokens.
2. Support Daily and Weekly habit scheduling, check-ins, calendar status and streak math.
3. Deliver goals, achievements, journal entries and in-app notification reminders.
4. Integrate Google Gemini for personalized habit suggestions and journal mood analysis.
5. Generate on-demand PDF progress reports without permanently storing report files.
6. Offer optional Google Calendar OAuth sync with encrypted token storage.
7. Expose a documented REST API and maintain automated pytest + frontend smoke coverage.

1.5 Scope

In scope (implemented):

- Web SPA (React/Vite) and REST API (FastAPI) with SQLite via SQLAlchemy + Alembic.
- Auth, profile/settings, habits, check-ins, calendar, analytics, goals, achievements, journal, notifications.
- AI Insights (habit suggestions + mood analysis), PDF export, Google Calendar connect/sync.
- Automated backend tests and Node smoke scripts under frontend/scripts.

Out of scope / not claimed as implemented:

- Native mobile apps, email/push notification delivery channels.
- Medical diagnosis from mood analysis.
- Automatic live Google OAuth consent without operator-configured Cloud credentials.
- Permanent server-side storage of AI outputs or PDF binaries.

1.6 Target Users

- Individuals seeking structured habit and goal tracking.
- Students and professionals documenting personal growth workflows.
- Reviewers evaluating full-stack engineering (API design, security, AI integration).

1.7 Proposed Solution

Habit Hero uses a layered architecture: React pages consume Axios API modules; FastAPI routers delegate to service modules; SQLAlchemy models persist per-user data; Gemini and Google Calendar are isolated behind backend services so secrets never reach the browser.

1.8 Key Differentiators

- Unified habit + journal + goals + analytics workspace with JWT isolation.
- Structured Gemini JSON outputs validated by Pydantic schemas.
- Fernet-encrypted Google OAuth tokens and HMAC-signed OAuth state.
- Deterministic scheduling/streak utilities shared by calendar and analytics.
- On-demand PDF composed from live analytics aggregates.

1.9 Expected Outcomes

Users can register, build habit routines, inspect calendar and analytics, reflect in a journal, request AI coaching insights, export PDF reports and optionally mirror habits to Google Calendar—demonstrating production-oriented full-stack skills.

2. Complete Technology Stack

2.1 Frontend Technologies

Technology	What it is	Why / Where in Habit Hero
React 19	UI component library	All pages under src/pages; contexts for auth/habits
Vite 8	Frontend build tooling	dev/build/preview scripts in package.json
JavaScript (ES modules)	Application language	src/**/*.jsx and services
HTML5 / CSS3	Markup and styling	index.html, page CSS, Tailwind utilities
React Router DOM 7	Client routing	App.jsx route table + ProtectedRoute
Context API	Global client state	AuthContext, HabitContext
Axios	HTTP client	apiClient.js + domain API modules
Lucide React	Icon set	Sidebar and page UI icons
Recharts	Charts	Analytics/dashboard visualizations
date-fns	Date helpers	Calendar/journal date handling
Tailwind CSS 4	Utility CSS	@tailwindcss/vite integration

2.2 Backend Technologies

Technology	Responsibility in Habit Hero
Python 3	Backend runtime
FastAPI	REST API, OpenAPI/Swagger, dependency injection
Uvicorn	ASGI server
SQLAlchemy 2	ORM models and sessions
Alembic	Schema migrations (4 revisions)
Pydantic v2 / pydantic-settings	Request/response validation and Settings
SQLite	Local relational store (habit_hero.db)
python-jose	JWT encode/decode (HS256)
passlib + bcrypt	Password hashing
google-genai	Gemini generate_content / structured JSON
httpx	Google OAuth/Calendar HTTP calls
cryptography Fernet	Encrypt Google tokens at rest (token_crypto.py)
ReportLab	In-memory PDF composition
Pillow	Avatar image processing

Pytest + httpx TestClient	API/unit tests
pypdf	PDF test assertions (tests only)

2.3 Development Tools (verified usage)

Tool	Usage
Git / GitHub	Version control; remote Habit-Hero- on main
PowerShell	Windows setup commands in README
Swagger UI / OpenAPI	FastAPI /docs when server running
Google Cloud Console	Required for live Calendar OAuth credentials
Cursor / VS Code class editors	Development environment for this codebase
Node.js + npm	Frontend install, Vite build, smoke scripts

Versions referenced from frontend/package.json and backend/requirements.txt at documentation time. No unused speculative frameworks are listed.

3. System Architecture

3.1 Overview

Habit Hero follows a classic SPA ↔ REST API ↔ ORM ↔ database pattern with optional external integrations invoked only from the backend service layer.

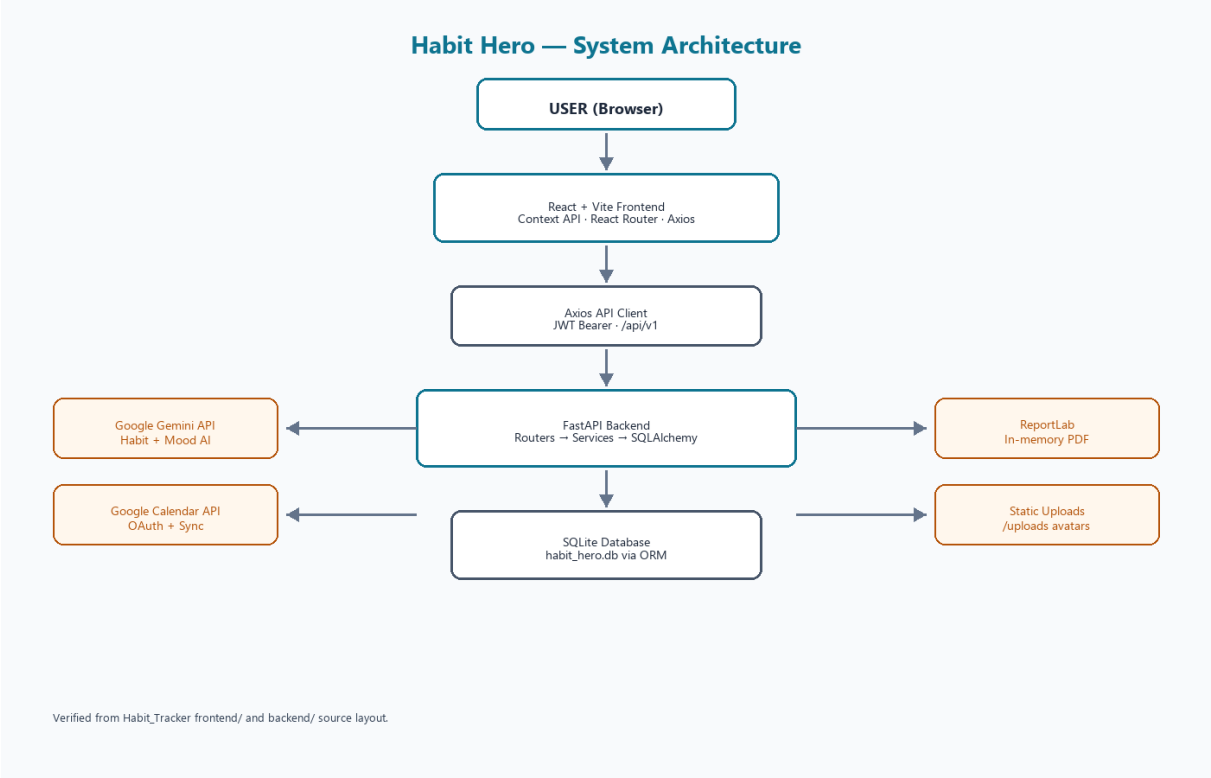


Figure 3.1 — System architecture

3.2 Frontend Architecture

- Entry: src/main.jsx mounts App.jsx.
- Routing: public /login and /signup; authenticated MainLayout routes for product pages.
- State: AuthProvider wraps session/profile/settings; HabitProvider loads domain data when authenticated.
- API boundary: src/services/api/* centralizes Axios calls and token refresh.

3.3 Backend Architecture

- app/main.py creates FastAPI app, CORS, uploads mount, lifespan scheduler hook.
- app/api/v1/router.py aggregates endpoint routers under settings.api_v1_prefix (default /api/v1).
- Endpoint modules call service modules; there is no separate repository package—services use SQLAlchemy Session directly.
- app/core holds config, security, scheduler; app/models and app/schemas define persistence and contracts.

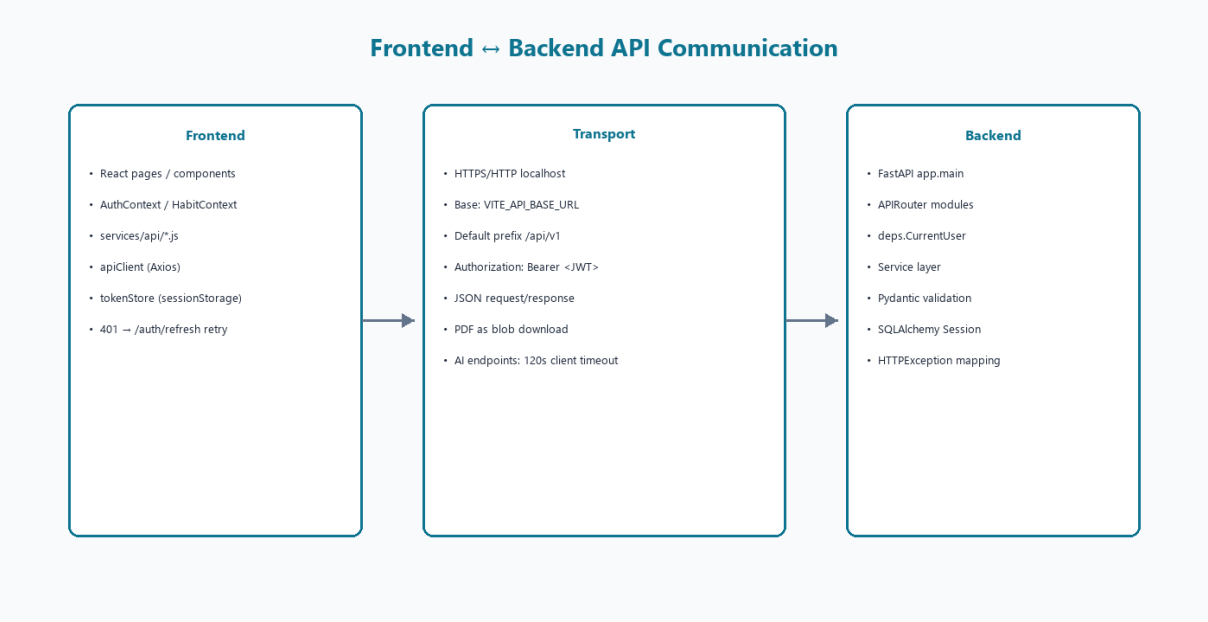


Figure 3.2 — API communication

3.4 Folder Structure (project root)

```
Habit_Tracker/
  frontend/     React + Vite SPA
  backend/      FastAPI + SQLAlchemy + Alembic + tests
  docs/         Plans and validation reports
  README.md     Root setup overview
  .gitignore    Excludes .env, *.db, node_modules, .venv, dist
```

3.5 External Integrations

- Google Gemini via gemini_service (backend-only GEMINI_* settings).
- Google Calendar via google_calendar_service + Fernet token_crypto.
- ReportLab PDF via pdf_report_service streamed to the client.

4. Complete Frontend Pages

Routes are defined in frontend/src/App.jsx. Screenshots of a running UI were not stored in the repository; therefore this chapter documents pages from source rather than fabricated images. Asset hero.png exists under src/assets but is not referenced by routed pages.

4.1 Dashboard (`/`) — Dashboard.jsx

Purpose: Authenticated home overview of habit productivity.

Features

- Welcome header and summary statistics
- Today's habits, weekly progress, category breakdown
- Streak and monthly overview widgets; progress trend; next goal
- Data primarily via analytics hooks (useDashboardAnalytics) and HabitContext

Backend APIs used

GET /analytics/* aggregates and habit/goal context data

Benefits

Provides a focused UX surface for the corresponding domain module while keeping secrets and business rules on the server.

4.2 Habit Management (`/habits`) — Habits.jsx

Purpose: Create, edit, archive/restore and check in habits.

Features

- Tabs: all/active/completed/archived; search and filters
- List/grid presentations; add/edit modal; delete confirmation
- Categories: Health, Work, Learning, Productivity, Mental Health
- Frequencies: Daily and Weekly (with week_day)

Backend APIs used

CRUD /habits and nested /habits/{id}/checkins

Benefits

Provides a focused UX surface for the corresponding domain module while keeping secrets and business rules on the server.

4.3 Calendar (`/calendar`) — Calendar.jsx

Purpose: Monthly and daily habit status with day notes. `/progress` redirects here.

Features

- Monthly grid from GET /calendar/monthly
- Daily summary with completed/pending/missed statuses
- Day notes CRUD via /calendar/notes
- Monthly statistics and legend

Backend APIs used

/calendar/monthly, /daily, /notes

Benefits

Provides a focused UX surface for the corresponding domain module while keeping secrets and business rules on the server.

4.4 Analytics (`/analytics`) — Analytics.jsx

Purpose: Performance analytics with date-range filtering and PDF export.

Features

- Tabs: overview, habits, categories, trends, insights
- DateRangeFilter drives analytics API queries
- PDFExportButton opens modal for progress PDF download
- Charts via Recharts components under components/analytics

Backend APIs used

/analytics/* and GET /reports/progress/pdf

Benefits

Provides a focused UX surface for the corresponding domain module while keeping secrets and business rules on the server.

4.5 Goals (`/goals`) — Goals.jsx

Purpose: Goal lifecycle plus achievements panel.

Features

- Create/edit/delete goals; progress visualization
- Linked habits and target types supported by backend schemas
- Achievements list/summary from achievement APIs

Backend APIs used

/goals*, /achievements*

Benefits

Provides a focused UX surface for the corresponding domain module while keeping secrets and business rules on the server.

4.6 Journal (`/journal`) — Journal.jsx

Purpose: Mood-tagged journal entries and history.

Features

- Create/edit/delete entries with tags and optional habit link
- Stats, calendar cues, recent entries, history modal
- Supports later AI mood analysis which requires ≥3 entries in range

Backend APIs used

/journal*

Benefits

Provides a focused UX surface for the corresponding domain module while keeping secrets and business rules on the server.

4.7 AI Insights (`/ai-insights`) — AIInsights.jsx

Purpose: Landing page for Gemini-powered coaching features.

Features

- HabitSuggestions component → POST /ai/habit-suggestions
- JournalMoodAnalysis component → POST /ai/mood-analysis
- Add Habit action persists a suggestion through normal habit create
- Extended Axios timeout (120s) for AI calls in aiApi.js

Backend APIs used

/ai/habit-suggestions, /ai/mood-analysis

Benefits

Provides a focused UX surface for the corresponding domain module while keeping secrets and business rules on the server.

4.8 Settings (`/settings`) — Settings.jsx

Purpose: Preferences, reminders, Google Calendar, local data clear.

Features

- Account card links to /profile

- Streak goal target and eight reminder preference toggles
- GoogleCalendarSettings: connect/sync/disconnect/status
- Clear local STORAGE_KEY habit cache

Backend APIs used

/users/me/settings, /calendar/google/*

Benefits

Provides a focused UX surface for the corresponding domain module while keeping secrets and business rules on the server.

4.9 Profile (`/profile`) — Profile.jsx

Purpose: Profile editing, avatar upload/remove, password change, logout.

Features

- PATCH /users/me, avatar POST/DELETE, password PATCH
- Logout clears tokens and returns to /login

Backend APIs used

/users/me*

Benefits

Provides a focused UX surface for the corresponding domain module while keeping secrets and business rules on the server.

4.10 Authentication — Login.jsx / Signup.jsx

Purpose: Public auth pages with redirect-to-origin after login.

Features

- Signup enforces password complexity rules in UI
- ProtectedRoute gates MainLayout children
- JWT stored via tokenStore (memory + sessionStorage)

Backend APIs used

/auth/register, /auth/login, /auth/refresh, /auth/logout, /auth/me

Benefits

Provides a focused UX surface for the corresponding domain module while keeping secrets and business rules on the server.

4.11 Unrouted page note

frontend/src/pages/Progress/Progress.jsx exists but is not imported in App.jsx; navigation to /progress redirects to /calendar. It is therefore not part of the live route map.

5. Backend Modules

Backend responsibilities are organized as FastAPI endpoint modules plus service modules. There is no separate repository layer package in this codebase.

Module	Primary code	Responsibilities
Authentication	auth endpoints + auth_service + security.py	Register/login, JWT pair, refresh rotation, logout, /auth/me
User Profile	users endpoints + user_service	Profile, password, avatar (Pillow), settings flags
Habit Management	habits + habit_service	CRUD, archive/restore, safe delete-vs-archive
Habit Check-ins	checkins + checkin_service	Per-day completion with unique (habit_id, date)
Calendar	calendar + calendar_service + day_note_service	Monthly/daily/summary statuses and day notes
Analytics	analytics + analytics_service + streak utils	Overview, weekly/monthly, categories, best days, streaks
Goals	goals + goal_service + goal_progress	CRUD, summary, computed progress by target_type
Achievements	achievements + achievement_service	Catalogue evaluation and earned records
Journal	journal + journal_service	Entries, tags, moods, summaries
Notifications	notifications + notification_service + notification_generator	In-app inbox generation and read state
AI Habit Suggestions	ai + ai_habit_service + gemini_service	Structured Gemini suggestions with validation
AI Mood Analysis	ai + ai_journal_service + gemini_service	Journal range analysis with minimum entry gate
PDF Reports	reports + pdf_report_service	Streaming ReportLab PDF
Google Calendar	google_calendar + google_calendar_service + token_crypto	OAuth, encrypted tokens, habit event sync

5.1 Cross-cutting concerns

- Validation: Pydantic request models (extra=forbid on many writes).
- AuthZ: CurrentUser dependency isolates rows by user_id.
- Errors: HTTPException mapping in services/endpoints; Gemini errors classified in gemini_service.
- Security: bcrypt passwords, hashed refresh tokens, Fernet Google tokens, CORS from settings.

6. Complete REST API Documentation

Default API prefix is /api/v1 (Settings.api_v1_prefix). Interactive Swagger UI is available at http://127.0.0.1:<port>/docs when Uvicorn is running. Local development has successfully used port 8010 when port 8000 hit Windows socket permission issues; README examples often show 8000. Frontend .env.example documents VITE_API_BASE_URL pointing at 8010.

6.1 Endpoint catalogue

Method	Path	Purpose	Auth
GET	/api/v1/health	Health check	No
POST	/api/v1/auth/register	Create account	No
POST	/api/v1/auth/login	Login token pair	No
POST	/api/v1/auth/refresh	Rotate refresh + access	No
POST	/api/v1/auth/logout	Revoke refresh	No
GET	/api/v1/auth/me	Auth profile	Yes
GET/PATCH	/api/v1/users/me	Profile	Yes
POST/DELETE	/api/v1/users/me/avatar	Avatar	Yes
PATCH	/api/v1/users/me/password	Change password	Yes
GET/PATCH	/api/v1/users/me/settings	Settings	Yes
POST/GET	/api/v1/habits	Create/list habits	Yes
GET/PATCH/DELETE	/api/v1/habits/{id}	Habit detail/update/delete	Yes
PATCH	/api/v1/habits/{id}/archive|restore	Archive lifecycle	Yes
POST/GET	/api/v1/habits/{id}/checkins	Check-ins	Yes
GET/PATCH/DELETE	/api/v1/habits/{id}/checkins/...	Check-in by date/id	Yes
GET	/api/v1/calendar/monthly|daily|summary	Calendar views	Yes
CRUD	/api/v1/calendar/notes	Day notes	Yes
GET	/api/v1/calendar/google/status|connect|events	Google status/connect/list	Yes*
GET	/api/v1/calendar/google/callback	OAuth callback	No (signed state)
POST	/api/v1/calendar/google/disconnect|sync	Disconnect/sync	Yes
DELETE	/api/v1/calendar/google/events/{id}	Delete mapped event	Yes
GET	/api/v1/analytics/*	Overview/habit/weekly/monthly/categories/best-days/streaks	Yes
CRUD	/api/v1/goals	Goals + /summary	Yes
GET	/api/v1/achievements*	Earned + summary	Yes

CRUD	/api/v1/journal	Journal + /summary	Yes
GET/POST/PATCH/DELETE	/api/v1/notifications*	Inbox + generate + read	Yes
POST	/api/v1/ai/habit-suggestions	AI suggestions	Yes
POST	/api/v1/ai/mood-analysis	AI mood analysis	Yes
GET	/api/v1/reports/progress/pdf	PDF stream	Yes

** Google connect/status/events require JWT except the OAuth callback, which validates HMAC state.*

6.2 Representative AI schemas (Pydantic)

HabitSuggestionsRequest: count integer default 5, ge=1, le=5.

HabitSuggestionItem fields: name, description, category (HABIT_CATEGORIES), frequency (Daily|Weekly), reason, recommended_start_date (YYYY-MM-DD).

MoodAnalysisRequest: start_date, end_date (end ≥ start). MoodAnalysisData includes overall_mood, mood_summary, mood_patterns.positive/challenging, motivation_patterns, positive_observations, areas_to_improve, recommendations, summary, entries_analyzed, analysis_period.

6.3 Error handling patterns

- 401/403 for missing/invalid Bearer tokens on protected routes.
- 422 for Pydantic validation failures.
- AI: 503 not configured / overloaded; 429 rate limit; 504 upstream timeout; 502 invalid upstream response.
- Google Calendar: refuses operation when GOOGLE_* / TOKEN_ENCRYPTION_KEY incomplete.

7. Google Gemini AI Integration

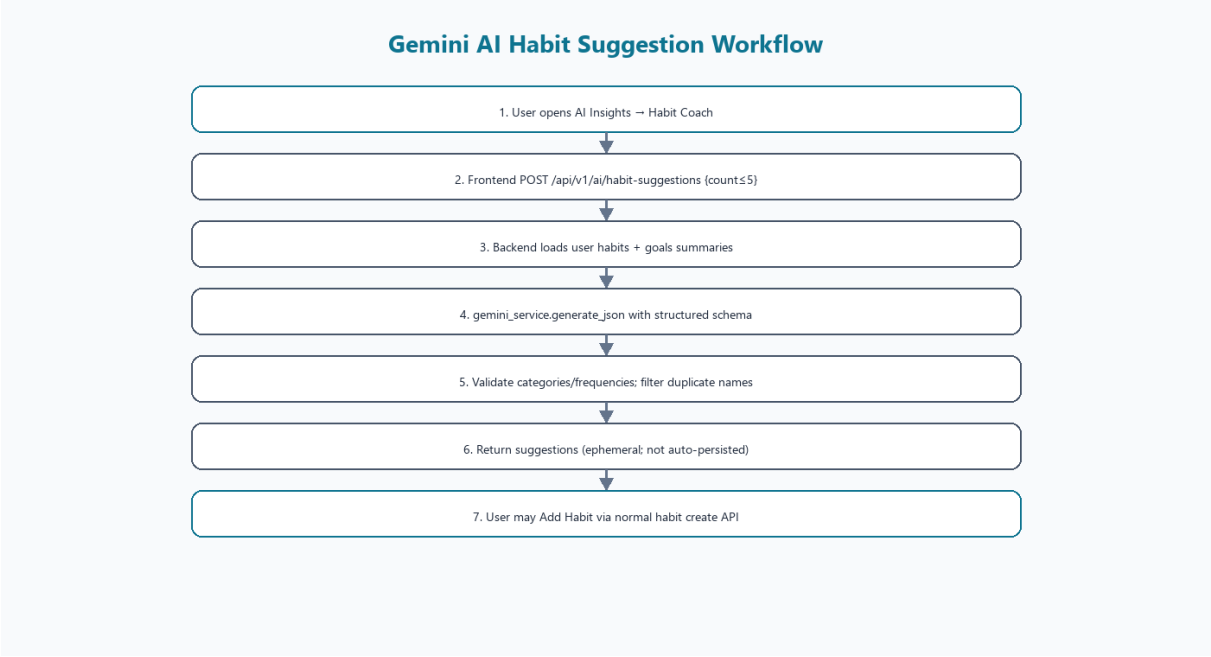


Figure 7.1 — Habit suggestion workflow

7.1 AI Habit Suggestions

8. User requests suggestions from AI Insights Habit Coach.
9. Frontend calls POST /api/v1/ai/habit-suggestions with count ≤ 5.
10. ai_habit_service loads habit and goal summaries for the authenticated user.
11. A prompt is composed and passed to gemini_service.generate_json with HabitSuggestionsLLMResponse schema.
12. Gemini returns structured JSON; backend validates categories/frequencies/dates.
13. Suggestions whose names collide with existing habits are filtered.
14. Response is ephemeral (not written as habits automatically).
15. User may add a suggestion through the standard habit create path.

7.2 AI Journal Mood Analysis

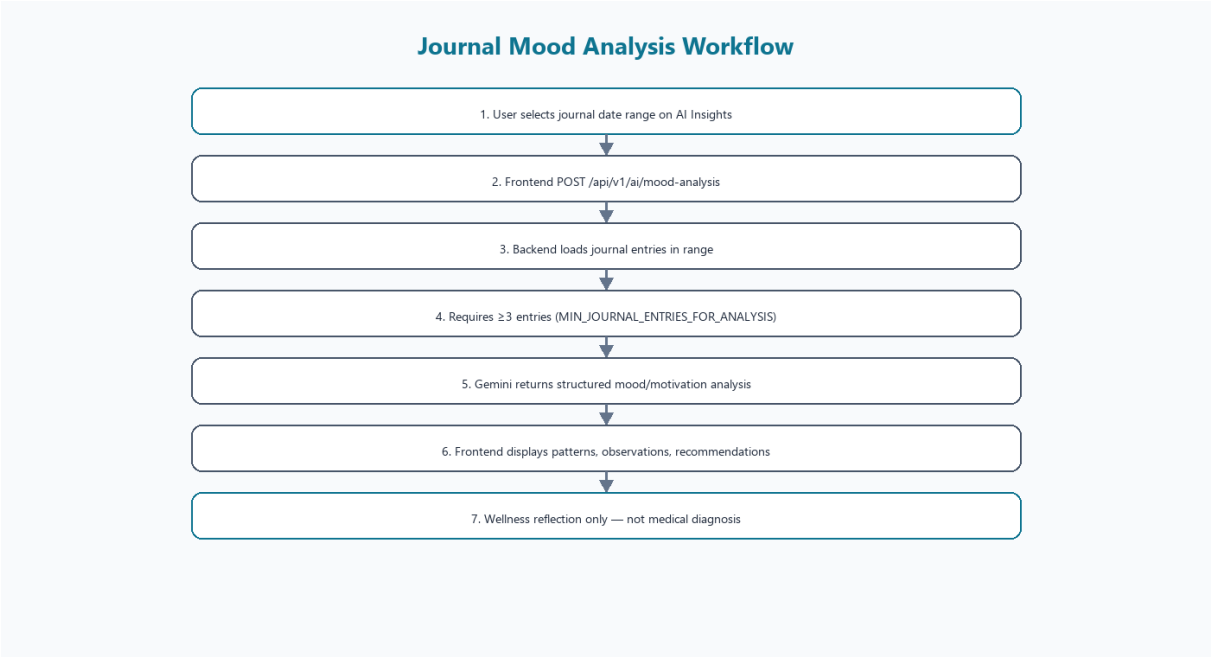


Figure 7.2 — Mood analysis workflow

16. User selects a date range.
17. Backend loads journal entries in range.
18. Requires at least MIN_JOURNAL_ENTRIES_FOR_ANALYSIS = 3 entries.
19. Gemini produces structured mood, motivation, observations and recommendations.
20. Frontend renders the analysis sections.
21. Content is wellness reflection, not medical diagnosis (product intent / UI copy).

7.3 Configuration (names only)

Variable	Purpose	Code default
GEMINI_API_KEY	Backend-only Google AI credential	empty → features disabled
GEMINI_MODEL	Model id for generate_content	gemini-3.8-flash
GEMINI_TIMEOUT_SECONDS	Client HttpOptions timeout (seconds→ms)	90 (allowed 5–180)

Credentials live in backend/.env and must never be exposed to Vite. google-genai Client is created with HttpOptions.timeout in milliseconds. Upstream 503 high-demand and 504 DEADLINE_EXCEEDED are classified into user-safe HTTP errors. Live Gemini latency/timeouts have been observed even with valid keys and listed models; mocked pytest does not prove live upstream availability.

8. Google Calendar Integration

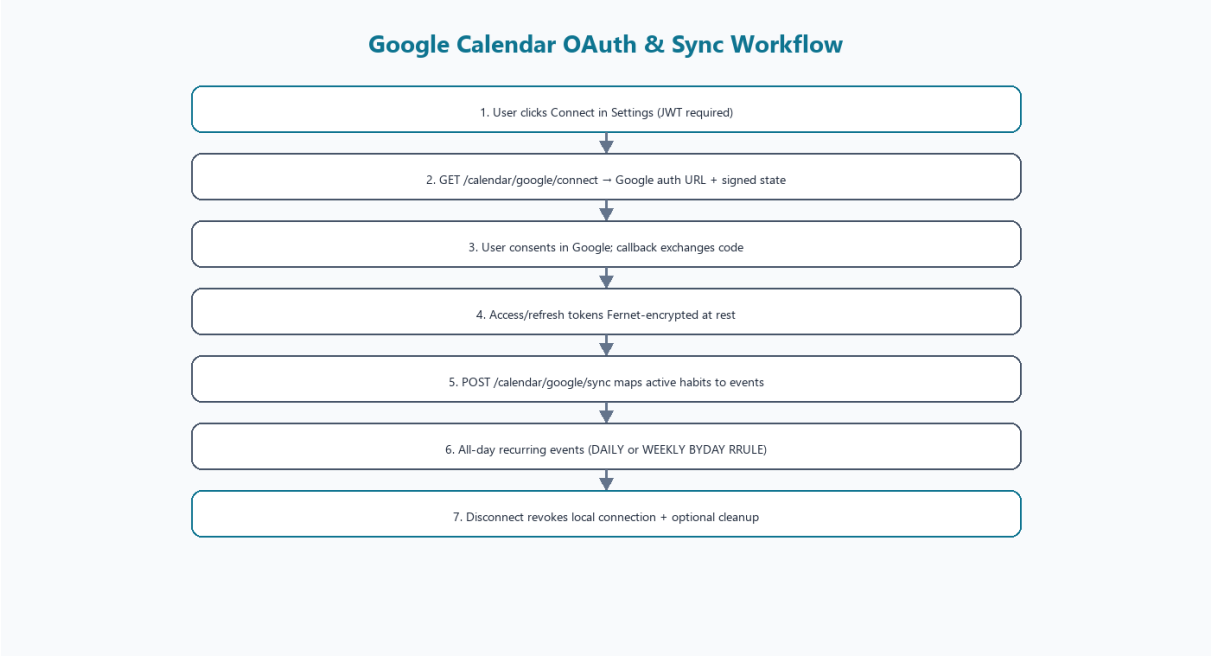


Figure 8.1 — OAuth and sync

8.1 Setup requirements

22. Create a Google Cloud project.
23. Enable Google Calendar API.
24. Configure OAuth 2.0 Client ID/Secret.
25. Set redirect URI to match GOOGLE_REDIRECT_URI.
26. Generate a stable Fernet TOKEN_ENCRYPTION_KEY.
27. Set FRONTEND_APP_URL for post-connect redirect to Settings.

8.2 Environment variable names

Variable	Purpose
GOOGLE_CLIENT_ID	OAuth client id
GOOGLE_CLIENT_SECRET	OAuth client secret
GOOGLE_REDIRECT_URI	Backend callback URL
TOKEN_ENCRYPTION_KEY	Fernet key for access/refresh token ciphertext
FRONTEND_APP_URL	Where callback redirects after success

TOKEN_ENCRYPTION_KEY must remain stable while encrypted rows exist; rotating it without re-auth renders stored tokens unreadable.

8.3 Runtime behavior

- Connect builds Google auth URL with HMAC-SHA256 signed state (≈10 minute expiry) bound to user id.
- Callback exchanges code, fetches userinfo, stores encrypted tokens in google_calendar_connections.
- Sync creates/updates/deletes mappings in habit_google_events for non-archived habits.
- Events are all-day because the habit model has no wall-clock time field.
- RRULE: FREQ=DAILY for daily habits; FREQ=WEEKLY;BYDAY=.. for weekly habits (Sun=0..Sat=6).
- API responses intentionally omit raw OAuth tokens.
- Live consent still requires operator-supplied Cloud credentials (documented limitation).

9. PDF Report Generation

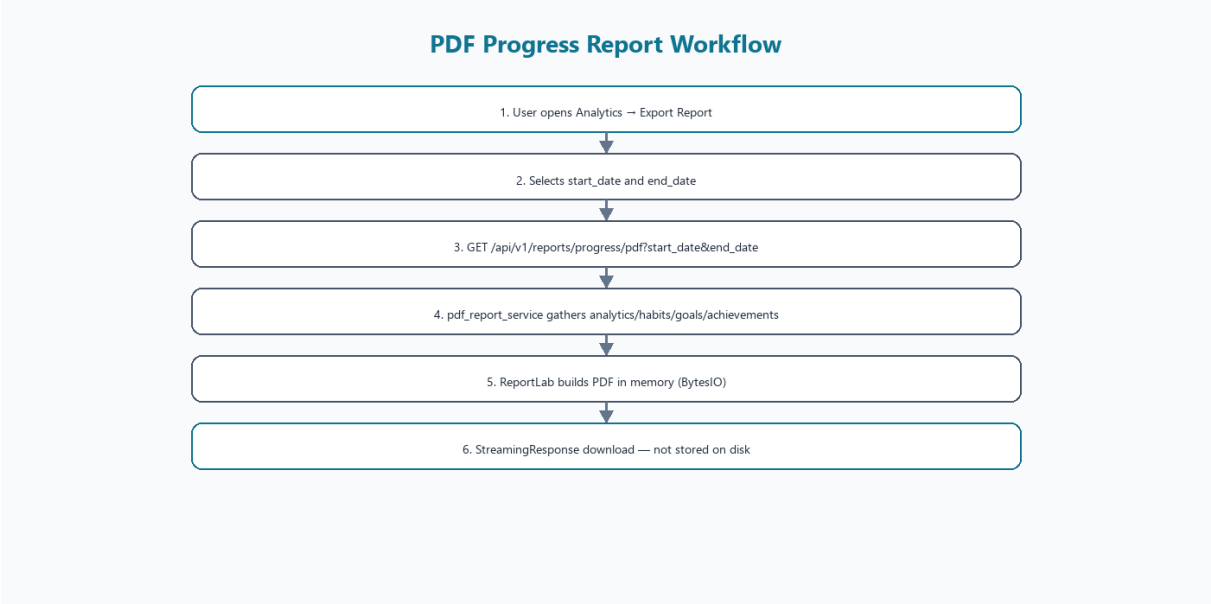


Figure 9.1 — PDF workflow

Endpoint GET /api/v1/reports/progress/pdf accepts authenticated start_date and end_date query params. pdf_report_service.build_progress_pdf composes a ReportLab document in memory and returns bytes. FastAPI streams them via StreamingResponse with Cache-Control: no-store. Reports are not persisted on disk. The PDF builder does not call Gemini.

9.1 Report sections (from code)

28. User Information
29. Overall Habit Summary
30. Individual Habit Performance
31. Streak Analysis
32. Calendar Progress
33. Category-wise Performance
34. Goal Progress
35. Achievements
36. Final Summary

Frontend PDFExport modal (components/reports/PDFExport.jsx) downloads a blob named like HabitHero_Progress_Report_*.pdf.

10. Database Design

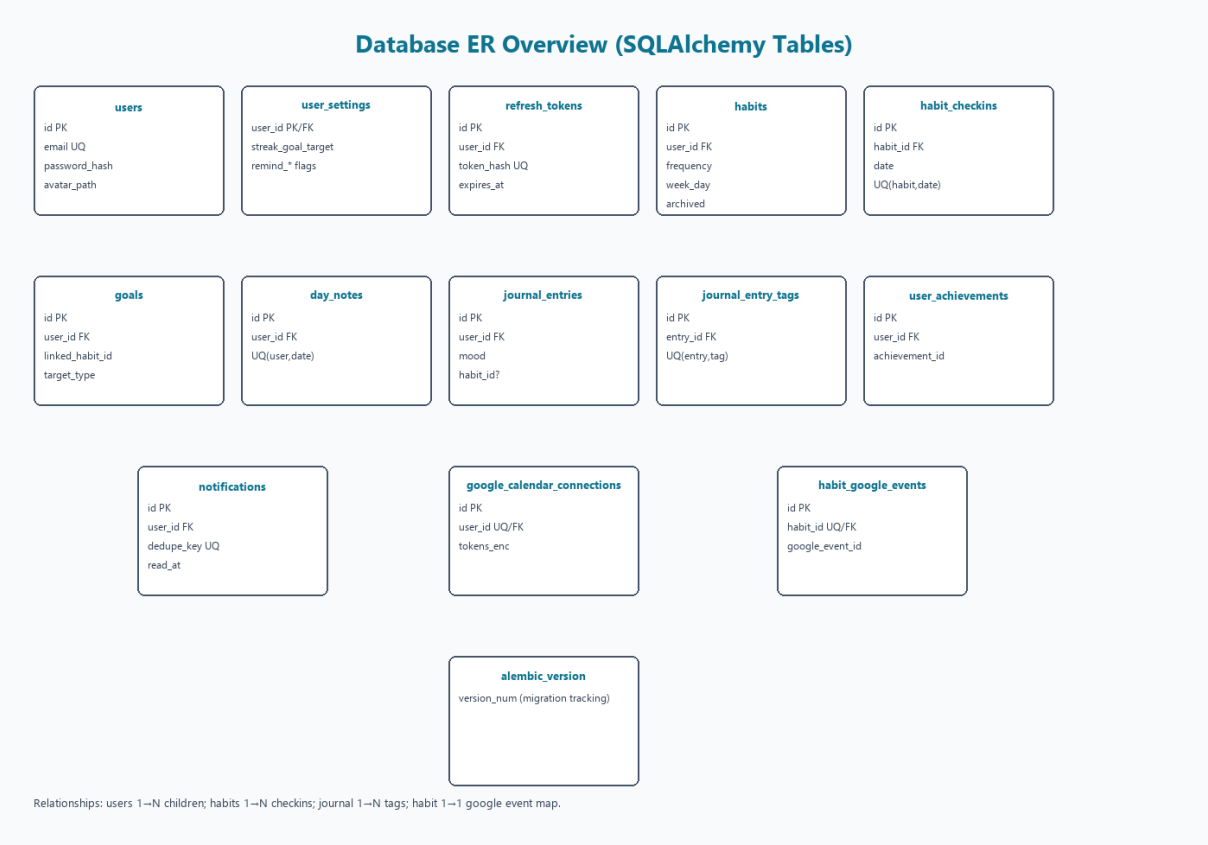


Figure 10.1 — ER overview

Persistence uses SQLAlchemy ORM against SQLite (DATABASE_URL, commonly sqlite:///./habit_hero.db). Alembic tracks schema via alembic_version. Four migration revisions create the tables below.

Table	Purpose	PK	Important columns / constraints
users	Accounts	id	email unique; password_hash; avatar_path
user_settings	Prefs / reminder flags	user_id PK/FK	streak_goal_target; remind_* integers
refresh_tokens	Refresh token hashes	id	token_hash unique; expires_at; revoked_at
habits	Habit definitions	id	frequency; week_day; archived; FK user
habit_checkins	Daily completions	id	UQ(habit_id,date); completed 0/1
goals	Personal goals	id	target_type/value; linked_habit_id
day_notes	Calendar notes	id	UQ(user_id,date)
journal_entries	Journal	id	mood; optional habit_id

journal_entry_tags	Tags	id	UQ(journal_entry_id,tag)
user_achievements	Unlocked badges	id	UQ(user_id,achievement_id)
notifications	In-app inbox	id	UQ(user_id,dedupe_key)
google_calendar_connections	OAuth connection	id	user_id unique; token ciphertext
habit_google_events	Habit↔event map	id	habit_id unique; google_event_id
alembic_version	Migration pointer	version_num	Alembic bookkeeping

10.1 ORM interaction

Services obtain a Session via FastAPI dependencies, query mapped classes under app/models, and commit unit-of-work changes. Cascades are declared on relationships (e.g., user delete orphans).

11. Authentication and Security

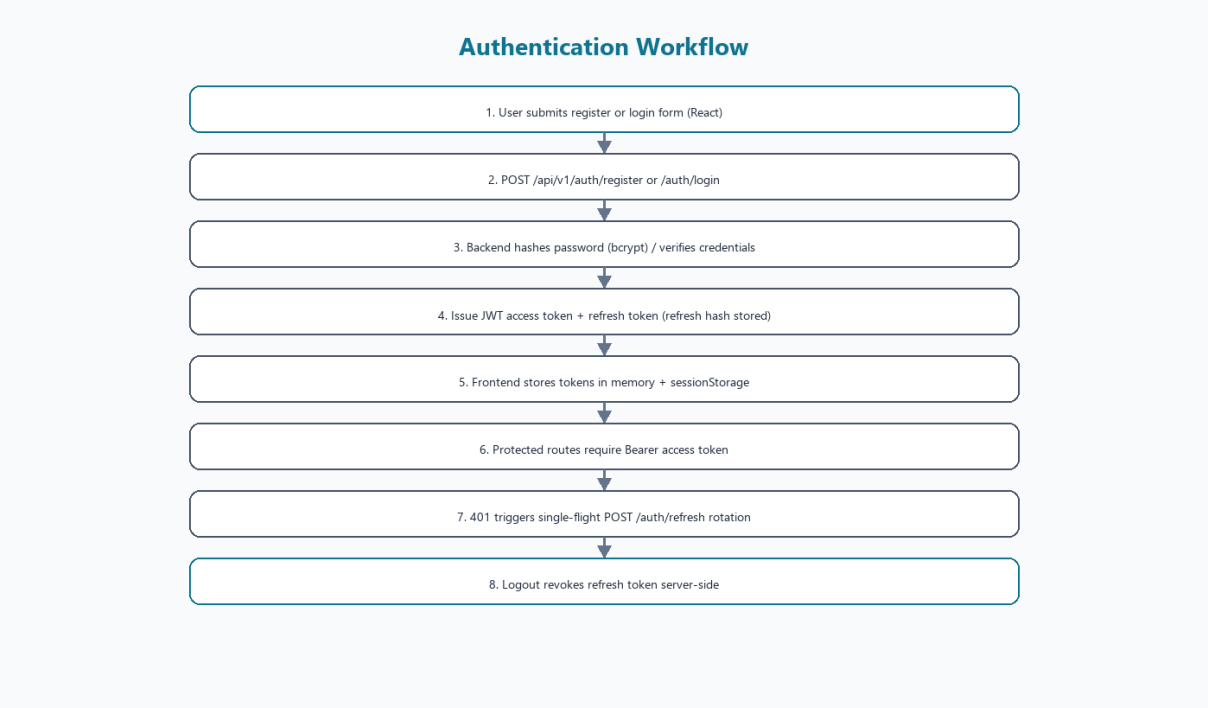


Figure 11.1 — Auth workflow

- Registration stores bcrypt password hashes (passlib CryptContext).
- Login returns JWT access (type=access) and refresh (type=refresh, jti).
- Refresh tokens persisted as SHA-256 hashes; rotation revokes prior row.
- Protected APIs depend on HTTPBearer + get_current_user.
- All domain queries scope by authenticated user_id.
- Pydantic validates payloads; many write schemas forbid unknown fields.
- Google OAuth state is HMAC-signed and user-bound.
- Google tokens encrypted with Fernet (TOKEN_ENCRYPTION_KEY).
- CORS origins configured via CORS_ORIGINS.
- Secrets belong in backend/.env (gitignored); only .env.example placeholders are committed.

11.1 Example environment placeholders

```
SECRET_KEY=change-me-in-production
GEMINI_API_KEY=
GEMINI_MODEL=gemini-3.8-flash
GEMINI_TIMEOUT_SECONDS=90
GOOGLE_CLIENT_ID=
GOOGLE_CLIENT_SECRET=
GOOGLE_REDIRECT_URI=http://127.0.0.1:8010/api/v1/calendar/google/callback
TOKEN_ENCRYPTION_KEY=
FRONTEND_APP_URL=http://127.0.0.1:5173
VITE_API_BASE_URL=http://127.0.0.1:8010/api/v1
```

12. Habit Tracking Algorithms

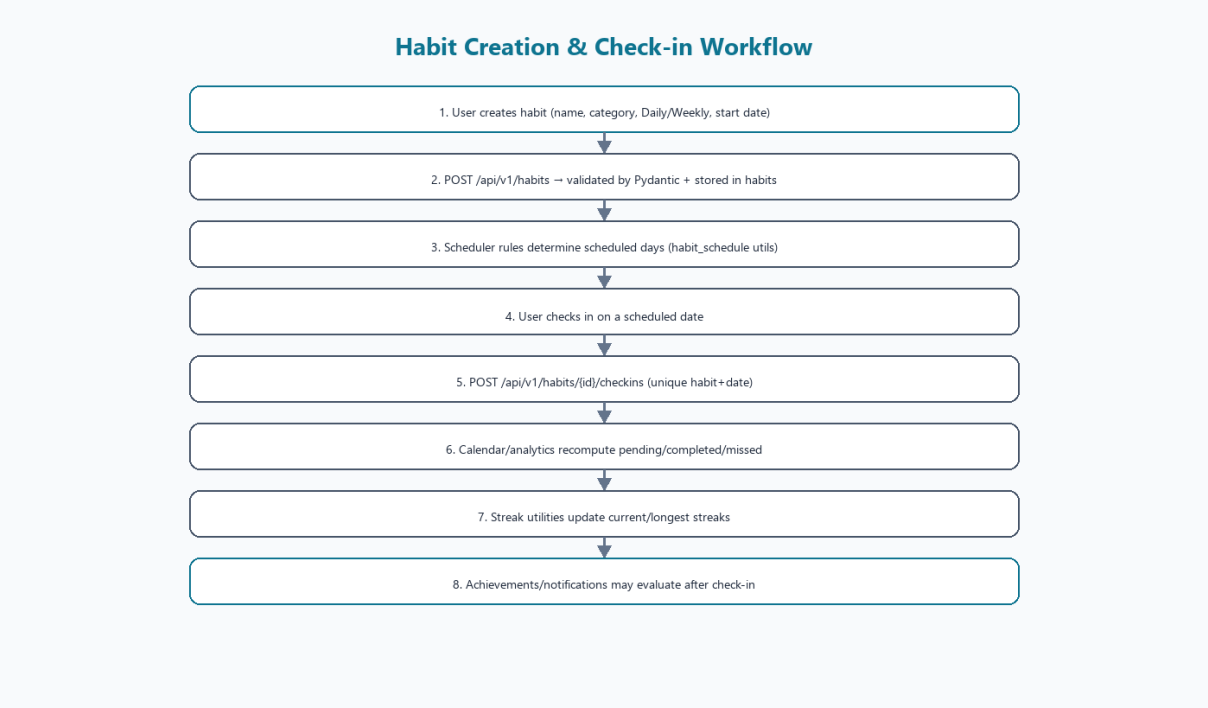


Figure 12.1 — Habit & check-in flow

12.1 Scheduling

utils/habit_schedule.py decides whether a habit is scheduled on a date. Daily habits are scheduled each day on/after start_date. Weekly habits match week_day where Sunday=0 … Saturday=6. Archived habits are excluded unless explicitly included.

12.2 Day status

calendar_service / analytics map a scheduled day to completed (check-in completed=1), pending (today or future), or missed (past incomplete). Unscheduled days are not_scheduled.

12.3 Completion percentage

```
completion_percentage = round((completed / scheduled) * 100)
# if scheduled <= 0: return 0
```

Implemented in app/utils/streak_calculations.py::completion_percentage.

12.4 Streaks

- Current streak walks backward from as_of.
- Days with no scheduled habits are skipped (do not break streak).
- Any incomplete scheduled day breaks the streak.
- Longest streak scans a lookback window (default 365 days).

12.5 Example

If a user has two daily habits and completes both on Mon–Wed, skips Thu (incomplete), then completes Fri, the current streak as of Friday is 1 (Thursday broke continuity). Unscheduled Sunday for a Mon-only weekly habit does not break a weekly streak calculation that skips unscheduled days.

13. Notifications and Reminders

Habit Hero implements an in-app notification inbox only. Email and push delivery are not implemented. notification_generator.py creates idempotent rows keyed by dedupe_key.

Type (generator)	Intent
habit_daily_reminder	Daily habit reminder
habit_weekly_reminder	Weekly habit reminder
habit_upcoming	Upcoming habit reminder
habit_overdue	Overdue habit reminder
goal_deadline / goal_overdue	Goal deadline reminders
streak_milestone	Streak milestone
weekly_summary	Weekly summary

User settings expose toggles for these reminder categories. An optional asyncio scheduler exists (NOTIFICATIONS_SCHEDULER_ENABLED, default false; interval default 900 seconds). Listing notifications defaults to refresh=true which regenerates before returning the inbox.

14. Complete User Workflow

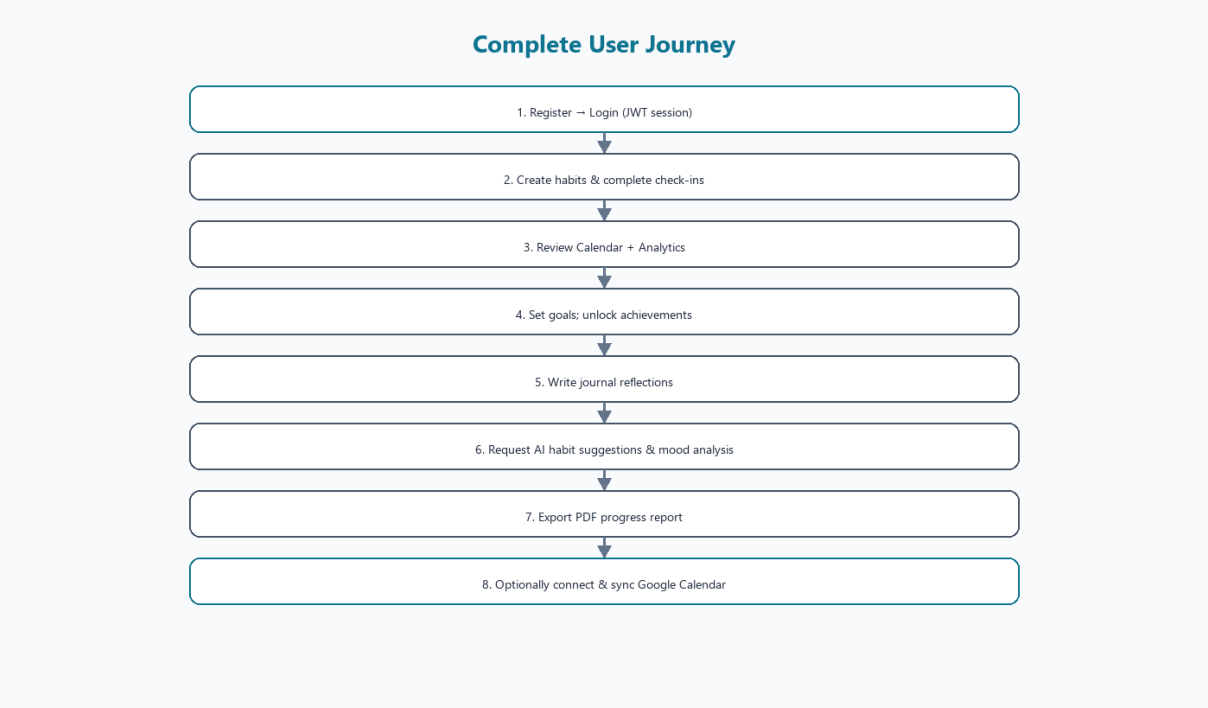


Figure 14.1 — User journey

37. Register an account
38. Log in and receive JWT session
39. Create Daily/Weekly habits
40. Complete check-ins on scheduled days
41. Review Calendar statuses and day notes
42. Monitor Analytics and streaks
43. Create goals and earn achievements
44. Write journal entries
45. Request AI habit suggestions
46. Run journal mood analysis
47. Export PDF progress report
48. Optionally connect and sync Google Calendar

15. Project Advantages

Advantage	Real-world value
Centralized habit management	One workspace for routines instead of spreadsheets
Daily + weekly scheduling	Supports both frequent and weekly cadences
Visual analytics/calendar	Makes consistency measurable
Personalized Gemini suggestions	Suggestions grounded in existing habits/goals
Journal mood insights	Structured reflection aids self-review
Goals + achievements	Motivation via targets and unlocks
Google Calendar sync	External visibility of routines
PDF reports	Shareable progress artifacts
JWT + user isolation	Multi-user safety on shared deployments
Modular API/services	Extensible backend for new features

16. Testing and Validation

FINAL_PROJECT_VALIDATION_REPORT.md (2026-11-11) recorded 200 pytest passed after Stage G. A later collect-only inspection of backend/tests reported 203 collected tests after additional Gemini error-classification coverage. Operators should re-run pytest locally for the latest number.

16.1 Backend pytest modules

- test_auth, test_users, test_habits, test_checkins, test_calendar
- test_analytics, test_goals, test_achievements, test_journal
- test_notifications (+ migration test)
- test_gemini_config, test_ai_habit_suggestions, test_ai_journal_analysis
- test_pdf_export, test_google_calendar

16.2 Distinguishing test types

Type	What it proves	What it does not prove
Mocked pytest (Gemini/Google HTTP)	App error handling & contracts	Live upstream availability
Live smoke scripts (frontend/scripts)	End-to-end against local API when run	CI guarantee unless executed
npm run build	Frontend bundles	Runtime UX quality
Manual browser checks	UI presence of routes/panels	Full regression alone

16.3 Known testing limitations

- Live Gemini may return 504/503 under load.
- Live Google OAuth requires real Cloud credentials + browser consent.
- Notification scheduler disabled by default in config.

17. Installation and Execution (Windows PowerShell)

17.1 Prerequisites

- Python 3.12+ recommended
- Node.js + npm
- Git
- Optional Google Cloud project for Calendar

17.2 Backend

```
cd E:\Habit_Tracker\backend
python -m venv .venv
.\.venv\Scripts\Activate.ps1
pip install -r requirements.txt
copy .env.example .env
# Edit .env — set SECRET_KEY and optional integrations (never commit secrets)
$env:PYTHONPATH = (Get-Location).Path
alembic upgrade head
uvicorn app.main:app --reload --host 127.0.0.1 --port 8010
```

Port note: README historically documents 8000. On some Windows setups port 8000 raised socket permission errors; 8010 was verified successfully. Align VITE_API_BASE_URL and GOOGLE_REDIRECT_URI with the chosen port. Swagger: http://127.0.0.1:8010/docs

17.3 Frontend

```
cd E:\Habit_Tracker\frontend
npm install
# Ensure .env has VITE_API_BASE_URL=http://127.0.0.1:8010/api/v1
npm run dev
```

App: http://localhost:5173

17.4 Troubleshooting

- 401 loops: check token refresh and SECRET_KEY stability.
- AI timeouts: inspect backend logs for Gemini 504/503; configuration may be correct while upstream is slow.
- Google connect disabled: ensure all GOOGLE_* and TOKEN_ENCRYPTION_KEY are set.
- CORS errors: include the Vite origin in CORS_ORIGINS.

18. GitHub and Version Control

Item	Verified value
Remote	https://github.com/vishnuk29072004-prog/Habit-Hero-.git
Default branch	main
Recent commits (sample)	Initial commit; AI/PDF/Calendar feature commit; validation doc commit
.gitignore	Excludes .env, *.db, .venv, node_modules, dist, caches

Development workflow: feature work locally → pytest/smoke → commit without secrets → push main. Do not commit backend/.env or frontend/.env.

19. Limitations

- Gemini upstream latency/timeouts (504 DEADLINE_EXCEEDED) and occasional 503 high demand.
- Live Google OAuth requires operator Cloud credentials and browser consent.
- Calendar sync uses all-day recurring events (no per-habit time-of-day in schema).
- AI outputs and PDFs are generated dynamically and not permanently stored as artifacts.
- Notifications are in-app only (no email/push).
- Notification scheduler disabled by default.
- Progress.jsx page exists but is not routed.
- Single SQLite file is suitable for local/demo deployments; horizontal scaling would need further work.

These are current limitations, not future roadmap items.

20. Future Enhancements (not implemented)

- Native mobile applications
- Deeper AI personalization and scheduled recommendations
- Email and push notification channels
- Social habit challenges and shared goals
- Advanced analytics / additional report formats
- Cloud deployment automation
- Additional calendar providers

Clearly labeled as future work — not present features.

21. Conclusion

Habit Hero demonstrates a complete full-stack personal growth platform: a React/Vite client, a FastAPI service layer, SQLite persistence with Alembic migrations, JWT security, analytics and streak algorithms, Gemini-powered coaching features, ReportLab PDF export, and Google Calendar OAuth sync with Fernet-encrypted tokens. The architecture keeps secrets on the server, validates inputs with Pydantic, isolates user data, and backs critical paths with automated tests while honestly documenting live third-party limitations.

As a portfolio and academic artifact, the project evidences API design, ORM modeling, authentication, frontend state management, third-party integration discipline, and documentation hygiene suitable for technical interviews and project reviews. Future enhancements can extend delivery channels and deployment topology without abandoning the modular service boundaries already in place.

22. Appendices

Appendix A — Diagram index

A.1 docs/diagrams/01_system_architecture.png

A.2 docs/diagrams/02_api_communication.png

A.3 docs/diagrams/03_database_er.png

A.4 docs/diagrams/04_auth_workflow.png

A.5 docs/diagrams/05_habit_checkin_workflow.png

A.6 docs/diagrams/06_gemini_habit_suggestions.png

A.7 docs/diagrams/07_mood_analysis_workflow.png

A.8 docs/diagrams/08_google_calendar_oauth.png

A.9 docs/diagrams/09_pdf_report_workflow.png

A.10 docs/diagrams/10_user_journey.png

Appendix B — Key source paths

- frontend/src/App.jsx — routes
- frontend/src/services/api/ — Axios modules
- backend/app/api/v1/endpoints/ — routers
- backend/app/services/ — business logic
- backend/app/models/ — SQLAlchemy tables
- backend/alembic/versions/ — migrations
- docs/FINAL_PROJECT_VALIDATION_REPORT.md — validation snapshot

Appendix C — References

- Project README.md and backend/README.md
- FastAPI / OpenAPI Swagger UI at runtime /docs
- Google Gemini API via google-genai SDK
- Google Calendar API OAuth 2.0 documentation
- ReportLab documentation
- GitHub repository Habit-Hero-

Appendix D — Verification statement

This document was generated from inspection of the Habit_Tracker codebase. Secret values from .env files were not read into the document. Where live third-party behavior could not be permanently guaranteed (Gemini availability, Google consent), limitations are stated explicitly.